MODULE 3 · MACHINE LEARNING · DAY 38

Ask the Neighbours

k-Nearest Neighbours

The first model that fits no equation at all.
It just remembers, and looks things up.

3	9	1
sections	worked pictures	line that decides it

Srinivasa Sai Chava · Boston University

WHAT CHANGES TODAY

Every model so far learned an equation

	what it learns	what it keeps	cost of training	cost of predicting
Linear regression	w and b	a few numbers	once	instant
Logistic regression	w and b	a few numbers	once	instant
k-Nearest Neighbours	nothing	the entire dataset	none at all	a search, every time

KNN does not learn. It memorises, and answers by looking things up.

To predict for a new applicant it finds the k most similar people it has already seen, and takes a vote. There is no formula to write down and no coefficient to interpret — which makes it the easiest model to explain and the easiest one to get badly wrong.

Because it keeps everything, the question 'which rows are closest?' becomes the whole model. Today is mostly about getting that one question right.

SECTION 1 OF 3

Ask the neighbours

The whole algorithm, and what k does to it

- find the k closest
- take a vote
- k = 1 to k = 101

FOUR STEPS

The entire algorithm

1 Pick k

how many neighbours get a vote

2 Measure the distance

from the new row to every row you stored

3 Take the k closest

and look up what happened to them

4 Vote

majority wins — or average, for regression

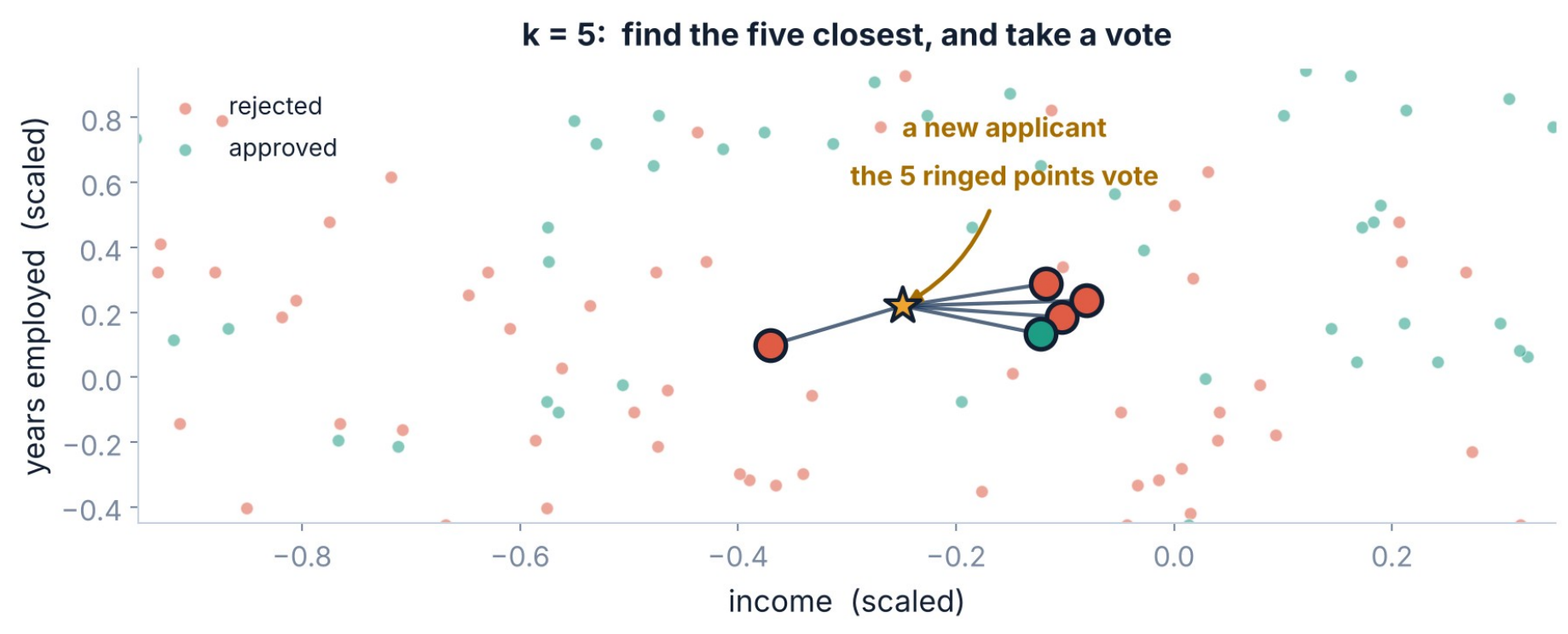
```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=15)
model.fit(X_train, y_train)        # this just stores the data
model.predict(X_new)               # this does all the work
```

Notice which line is slow. .fit() copies the data and stops. Every .predict() call searches the whole training set again.

THE PICTURE

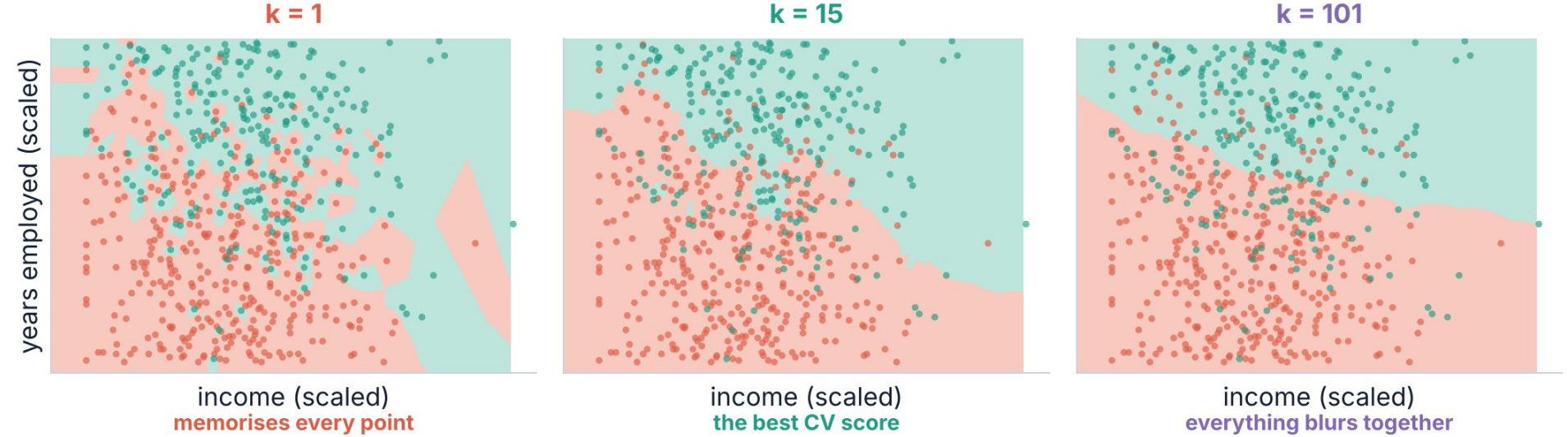
One new applicant, five neighbours, one vote



Three of the five nearest were rejected, so the model rejects. Change k to 3 and you may get a different answer from the same data.

THREE MODELS, SAME DATA

What k actually controls



k = 1 draws an island round every single point — that is memorising, and Day 36 told you what it costs. k = 101 asks so many neighbours that the local detail disappears.

SECTION 2 OF 3

Distance decides everything

The one line of code that makes or breaks this model

- income in rupees, service in years
- 4,496 : 1
- the answer flips

AND THAT IS THE PROBLEM

Distance is just Pythagoras

```
distance = sqrt( (income_A - income_B)^2
               + (years_A  - years_B)^2 )
```

Two applicants

A: income 44,000 · 18 years
B: income 44,200 · 0 years

distance = sqrt($200^2 + 18^2$) = 200.8

Eighteen years of service moved it by 0.8.

26,041

the spread of income

5.79

the spread of years

4,496 : 1

the ratio between them

So the distance is the income gap

Because income is measured in numbers about four and a half thousand times larger, it swamps the other column completely.

The years column is still in the data. The model simply cannot hear it.

No other model in this course cares about units. Linear and logistic regression just learn a smaller coefficient. KNN has no coefficient to adjust.

0 OF 5 AGAINST 5 OF 5

The same applicant, with and without one line

One applicant: income ₹44,000 · 18 years employed

DISTANCE ON THE RAW NUMBERS

income	years	approved?
43,938	3.6	no
43,892	2.6	no
44,200	11.6	no
44,209	0.8	no
44,275	12.1	no

vote 0 of 5 → REJECT

years of the five: 3.6, 2.6, 11.6, 0.8, 12.1

DISTANCE AFTER SCALING

income	years	approved?
43,643	18.3	yes
42,643	18.1	yes
41,617	18.0	yes
48,347	17.8	yes
48,522	18.1	yes

vote 5 of 5 → APPROVE

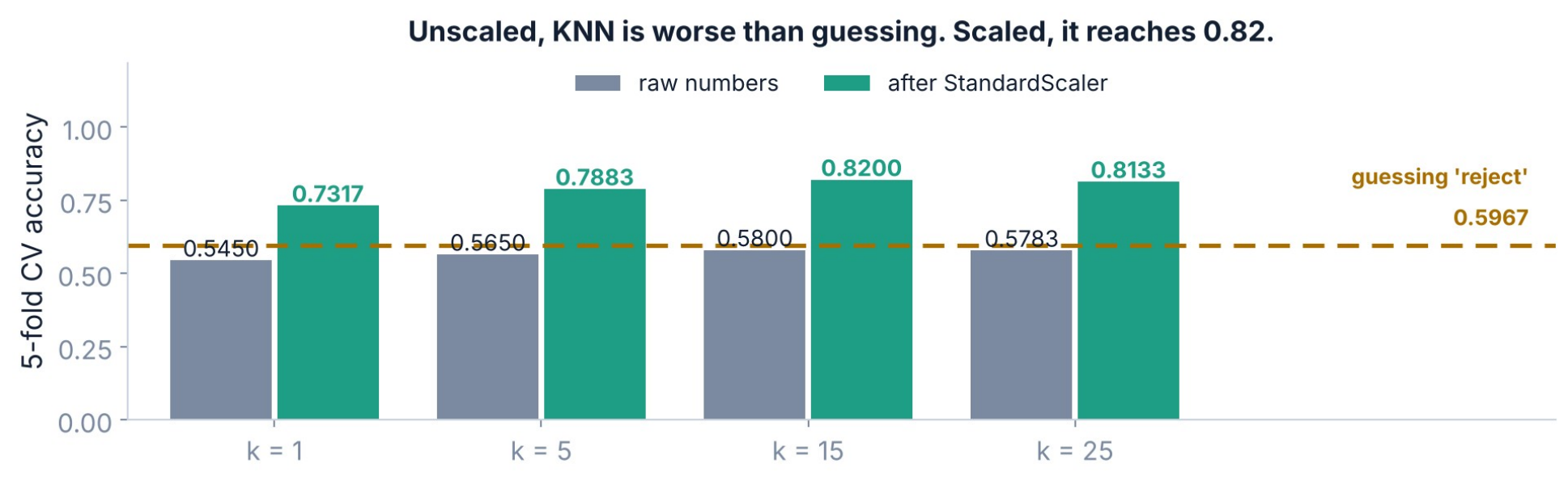
years of the five: 18.3, 18.1, 18.0, 17.8, 18.1

Same applicant. Same data. Same k. One line of code decides whether she gets the loan.

Unscaled, it found five people on the same salary and wildly different careers. Scaled, it found five on the same salary AND the same length of service.

600 LOAN APPLICATIONS

What that costs, measured



Unscaled, KNN scores below 'reject everybody' at every k we tried. One line of code is worth 24 points of accuracy.

THE FIX

So it always goes in a Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

knn = Pipeline([("scale", StandardScaler()),
                ("model", KNeighborsClassifier(n_neighbors=15))])

cross_val_score(knn, X, y, cv=cv).mean()        # 0.8200
```

Why a Pipeline and not just scaling first

Day 36. The scaler is refitted inside every fold, so no fold ever sees the rows it is about to be scored on.

WHICH SCALER?

StandardScaler is the usual choice. Use MinMaxScaler if your columns have hard limits, and RobustScaler if a few extreme values would otherwise stretch everything.

If you remember one thing from today: KNN without a scaler is not a model, it is the column with the biggest numbers.

SECTION 3 OF 3

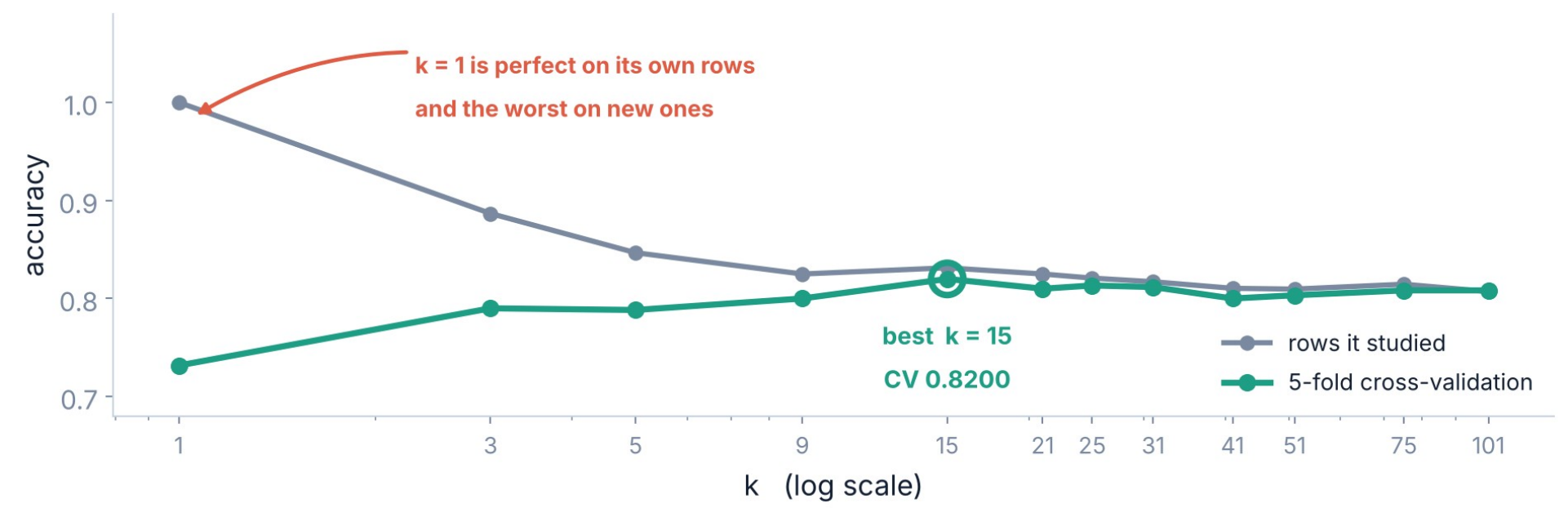
Choosing k

And the two things KNN is quietly bad at

- cross-validate it
- it never forgets
- and it hates wide data

THE WHOLE CURVE

Cross-validate k. Do not guess it.



At k = 1 the model is perfect on its own rows and the worst of all on new ones — the clearest picture of overfitting you will see this term.

SMALL K, LARGE K

How to read that curve

SMALL k

follows every point, including the noise

high variance — move one row and the boundary changes

k = 1 scored 1.0000 on its own rows and 0.7317 on new ones

LARGE k

averages over neighbours that are not really similar

high bias — the boundary goes smooth and stops fitting anything

k = 101 scored 0.8083, below the best

IN BETWEEN

k = 15 won here, at 0.8200

Rules of thumb exist (√n is a common one, ≈ 24 here) but they are only a starting point

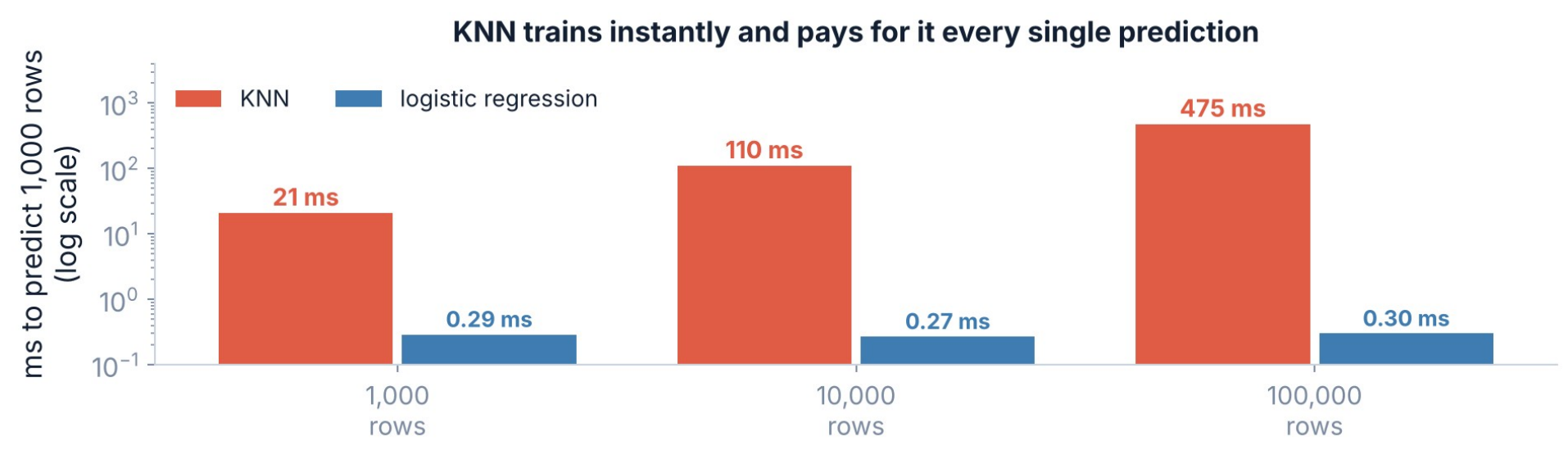
Use cross-validation

Two practical notes: use an odd k for two classes so a vote cannot tie · weights="distance" lets closer neighbours count for more

This is the bias-variance trade-off with a dial on it. Every model has one; KNN is the one where you can see it directly.

THE FIRST WEAKNESS

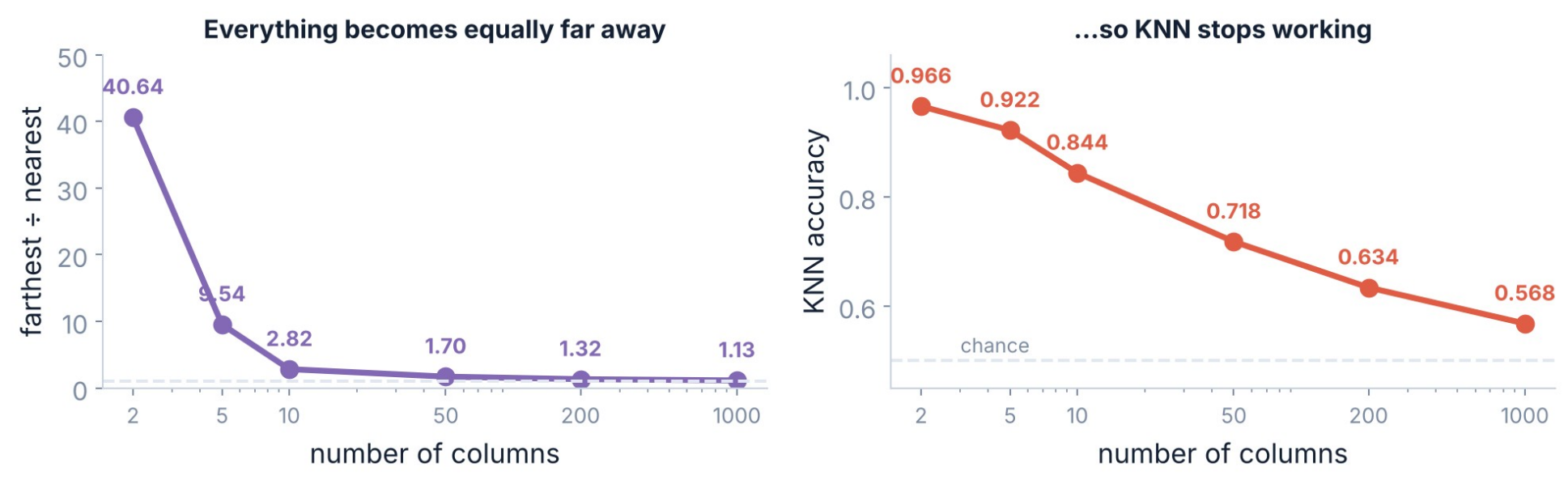
The bill arrives at prediction time



Training is free because nothing happens. Then every prediction searches the whole dataset — and for 100,000 stored rows that is 1,600 times slower than logistic regression.

THE SECOND WEAKNESS

And it falls apart on wide data

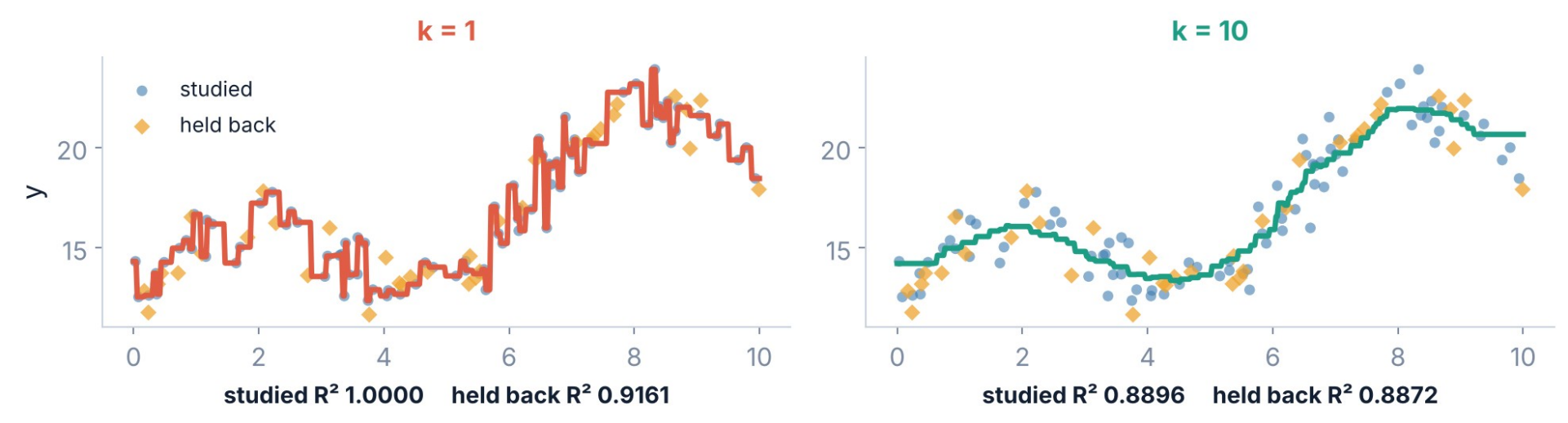


Both panels use data where only the first two columns carry any signal. The rest is noise — and noise still counts towards the distance.

This is the curse of dimensionality. Add enough columns and every point is about as far away as every other, so 'nearest' stops meaning anything.

SWAP THE VOTE FOR AN AVERAGE

It does regression too



KNeighborsRegressor averages the k neighbours instead of voting. The steps in the k = 1 line are the model copying single points — no equation, so no smooth curve.

AND WHEN NOT TO

When to reach for it

GOOD FIT

few columns, plenty of rows

the boundary is irregular and no equation would describe it

you want a baseline in two lines of code

recommendation-style problems: 'who is most like this customer?'

BAD FIT

many columns — text, images, anything wide

very large datasets, or predictions needed in milliseconds

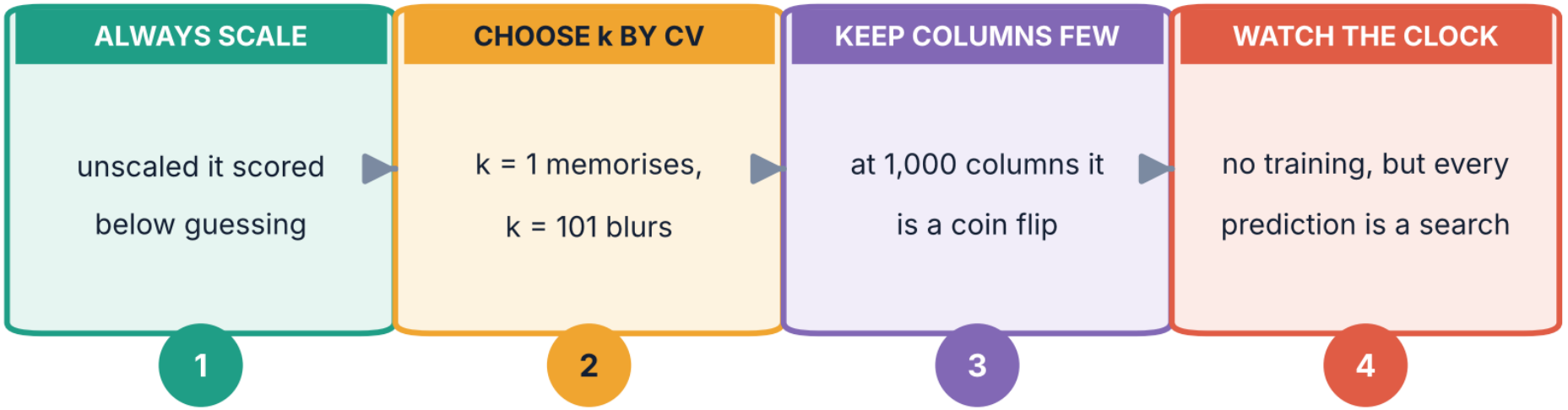
you need to explain WHY, to a regulator or a customer

missing values, which have no distance

On our loan data KNN scored 0.8200 against logistic regression's 0.8083 — a real win, on two columns. Add forty more columns and that result would reverse.

ALL OF TODAY ON ONE PAGE

The four habits



The simplest model in the course, and the one most often broken by a missing line of code.

Nothing here is about making KNN cleverer. Three of the four are about the fact that it measures distance, and distance is easy to break.

WHAT TO DO

Tonight's practice

A Work it out

6 questions

Distances and votes by hand, and what scaling does to them

B Find the mistake

3 questions

Three programs that forget what KNN needs

C Write the code

4 tasks

Scale it, cross-validate k, and compare against logistic regression

D Challenge

1 task

Write KNN from scratch and match scikit-learn exactly

Part D is short: KNN is the one model you can write correctly in about fifteen lines, and doing so makes the scaling problem impossible to forget.

WHAT TO REMEMBER

Four numbers from today

0 of 5 → 5 of 5 — the same applicant, rejected on raw numbers and approved after scaling

0.5800 → 0.8200 — KNN at k = 15 before and after one line of StandardScaler; 'reject everybody' scores 0.5967

1.0000 / 0.7317 — k = 1 on the rows it studied, and on rows it had never seen

40.64 → 1.13 — farthest ÷ nearest as the columns go from 2 to 1,000 — and accuracy falls to 0.5680

Next: decision trees — a model that explains itself.